

Authoring Content in Markdown

Starlight supports the full range of [Markdown](#) syntax in `.md` files as well as frontmatter [YAML](#) to define metadata such as a title and description.

Please be sure to check the [MDX docs](#) or [Markdoc docs](#) if using those file formats, as Markdown support and usage can differ.

Frontmatter

You can customize individual pages in Starlight by setting values in their frontmatter. Frontmatter is set at the top of your files between `---` separators:

```
src/content/docs/example.md
```

```
---  
title: My page title  
---
```

```
Page content follows the second `---`.
```

Every page must include at least a `title`. See the [frontmatter reference](#) for all available fields and how to add custom fields.

Inline styles

Text can be **bold**, *italic*, or ~~strikethrough~~.

```
Text can be bold, italic, or strikethrough.
```

You can [link to another page](#).

```
You can [link to another page](/getting-started/).
```

You can highlight `inline code` with backticks.

```
You can highlight `inline code` with backticks.
```

Images

Images in Starlight use [Astro's built-in optimized asset support](#).

Markdown and MDX support the Markdown syntax for displaying images that includes alt-text for screen readers and assistive technology.



```
![An illustration of planets and stars featuring the word "astro"]  
(https://raw.githubusercontent.com/withastro/docs/main/public/default-og-image.png)
```

Relative image paths are also supported for images stored locally in your project.

```
src/content/docs/page-1.md
```

```
![A rocketship in space](../assets/images/rocket.svg)
```

Headings

You can structure content using a heading. Headings in Markdown are indicated by a number of # at the start of the line.

How to structure page content in Starlight

Starlight is configured to automatically use your page title as a top-level heading and will include an “Overview” heading at top of each page’s table of contents. We recommend starting each page with regular paragraph text content and using on-page headings from <h2> and down:

```
---
title: Markdown Guide
description: How to use Markdown in Starlight
---

This page describes how to use Markdown in Starlight.

## Inline Styles

## Headings
```

Automatic heading anchor links

Using headings in Markdown will automatically give you anchor links so you can link directly to certain sections of your page:

```
---
title: My page of content
description: How to use Starlight's built-in anchor links
---

## Introduction

I can link to [my conclusion](#conclusion) lower on the same page.

## Conclusion

`https://my-site.com/page1/#introduction` navigates directly to my
Introduction.
```

Level 2 (<h2>) and Level 3 (<h3>) headings will automatically appear in the page table of contents.

Learn more about how Astro processes heading ids in [the Astro Documentation](#)

Asides

Asides (also known as “admonitions” or “callouts”) are useful for displaying secondary information alongside a page’s main content.

Starlight provides a custom Markdown syntax for rendering asides. Aside blocks are indicated using a pair of triple colons `:::` to wrap your content, and can be of type `note`, `tip`, `caution` or `danger`.

You can nest any other Markdown content types inside an aside, but asides are best suited to short and concise chunks of content.

Note aside

Note

Starlight is a documentation website toolkit built with [Astro](#). You can get started with this command:

```
npm create astro@latest -- --template starlight
```

```
:::note
Starlight is a documentation website toolkit built with [Astro]
(https://astro.build/). You can get started with this command:

```sh
npm create astro@latest -- --template starlight
```

:::
```

Custom aside titles

You can specify a custom title for the aside in square brackets following the aside type, e.g. `:::tip[Did you know?]`.

Did you know?

Astro helps you build faster websites with [“Islands Architecture”](#).

```
:::tip[Did you know?]
Astro helps you build faster websites with [“Islands Architecture”]
(https://docs.astro.build/en/concepts/islands/).

:::
```

Custom aside icons

You can specify a custom icon for the aside in curly brackets following the aside type or [custom title](#), e.g. `:::tip{icon="heart"}` or `:::tip[Did you know?]{icon="heart"}` respectively. The icon name must be set to the name of [one of Starlight's built-in icons](#).

♥ Tip

Astro helps you build faster websites with [“Islands Architecture”](https://docs.astro.build/en/concepts/islands/).

```
:::tip{icon="heart"}
Astro helps you build faster websites with [“Islands Architecture”]
(https://docs.astro.build/en/concepts/islands/).
:::
```

More aside types

Caution and danger asides are helpful for drawing a user’s attention to details that may trip them up. If you find yourself using these a lot, it may also be a sign that the thing you are documenting could benefit from being redesigned.

⚠ Caution

If you are not sure you want an awesome docs site, think twice before using [Starlight](#).

⚠ Danger

Your users may be more productive and find your product easier to use thanks to helpful Starlight features.

- Clear navigation
- User-configurable colour theme
- [i18n support](#)

```
:::caution
```

If you are not sure you want an awesome docs site, think twice before using [Starlight](/).

```
:::
```

```
:::danger
```

Your users may be more productive and find your product easier to use thanks to helpful Starlight features.

- Clear navigation
- User-configurable colour theme
- [i18n support](/guides/i18n/)

```
:::
```

Blockquotes

This is a blockquote, which is commonly used when quoting another person or document.

Blockquotes are indicated by a > at the start of each line.

```
> This is a blockquote, which is commonly used when quoting another
person or document.
```

```
>
```

```
> Blockquotes are indicated by a `>` at the start of each line.
```

Code blocks

A code block is indicated by a block with three backticks ``` at the start and end. You can indicate the programming language being used after the opening backticks.

```
// Javascript code with syntax highlighting.
var fun = function lang(l) {
  dateformat.i18n = require('./lang/' + l);
  return true;
};
```

```
```js
// Javascript code with syntax highlighting.
var fun = function lang(l) {
 dateformat.i18n = require('./lang/' + l);
 return true;
};
```
```

Expressive Code features

Starlight uses [Expressive Code](#) to extend formatting possibilities for code blocks. Expressive Code's text markers and window frames plugins are enabled by default. Code block rendering can be configured using Starlight's [expressiveCode configuration option](#).

Text markers

You can highlight specific lines or parts of your code blocks using [Expressive Code text markers](#) on the opening line of your code block. Use curly braces ({ }) to highlight entire lines, and quotation marks to highlight strings of text.

There are three highlighting styles: neutral for calling attention to code, green for indicating inserted code, and red for indicating deleted code. Both text and entire lines can be marked using the default marker, or in combination with `ins=` and `de=` to produce the desired highlighting.

Expressive Code provides several options for customizing the visual appearance of your code samples. Many of these can be combined, for highly illustrative code samples. Please explore the [Expressive Code documentation](#) for the extensive options available. Some of the most common examples are shown below:

- [Mark entire lines & line ranges using the {_} marker:](#)

```
function demo() {
  // This line (#2) and the next one are highlighted
  return 'This is line #3 of this snippet';
}
```

```
```js {2-3}
function demo() {
 // This line (#2) and the next one are highlighted
 return 'This is line #3 of this snippet';
}
```
```

- [Mark selections of text using the " " marker or regular expressions:](#)

```
// Individual terms can be highlighted, too
function demo() {
  return 'Even regular expressions are supported';
}
```

```
```js "Individual terms" /Even.*supported/
// Individual terms can be highlighted, too
function demo() {
 return 'Even regular expressions are supported';
}
```
```

- [Mark text or lines as inserted or deleted with ins or del:](#)

```
function demo() {
  console.log('These are inserted and deleted marker types');
  // The return statement uses the default marker type
  return true;
}
```

```
```js "return true;" ins="inserted" del="deleted"
function demo() {
 console.log('These are inserted and deleted marker types');
 // The return statement uses the default marker type
 return true;
}
```
```

- [Combine syntax highlighting with diff-like syntax:](#)

```
function thisIsJavaScript() {
  // This entire block gets highlighted as JavaScript,
  // and we can still add diff markers to it!
- console.log('Old code to be removed')
+ console.log('New and shiny code!')
}
```

```
```diff lang="js"
function thisIsJavaScript() {
 // This entire block gets highlighted as JavaScript,
 // and we can still add diff markers to it!
- console.log('Old code to be removed')
+ console.log('New and shiny code!')
}
```
```

Frames and titles

Code blocks can be rendered inside a window-like frame. A frame that looks like a terminal window will be used for shell scripting languages (e.g. bash or sh). Other languages display inside a code editor-style frame if they include a title.

A code block's optional title can be set either with a `title="..."` attribute following the code block's opening backticks and language identifier, or with a file name comment in the first lines of the code.

- [Add a file name tab with a comment](#)

```
my-test-file.js
```

```
console.log('Hello World!');
```

Markdown/MDX

Markdoc

```
```js
// my-test-file.js
console.log('Hello World!');
```
```

- [Add a title to a Terminal window](#)

Installing dependencies...

```
npm install
```

Markdown/MDX

Markdoc

```
```bash title="Installing dependencies..."
npm install
```
```

- [Disable window frames with frame="none"](#)

```
echo "This is not rendered as a terminal despite using the bash
language"
```

Markdown/MDX

Markdoc

```
```bash frame="none"
echo "This is not rendered as a terminal despite using the bash
language"
```
```

Details

Details (also known as “disclosures” or “accordions”) are useful to hide content that is not immediately relevant. Users can click a short summary to expand and view the full content.

Use the standard HTML [<details>](#) and [<summary>](#) elements in your Markdown content to create a disclosure widget.

You can nest any other Markdown syntax inside a `<details>` element.

Where and when is the Andromeda constellation most visible?

```
<details>
<summary>Where and when is the Andromeda constellation most visible?
</summary>

The [Andromeda constellation]
(<https://en.wikipedia.org/wiki/Andromeda_(constellation)>) is most visible
in the night sky during the month of November at latitudes between `+90°`
and `-40°`.

</details>
```

Footnotes

[Footnotes](#) are useful for adding brief references or clarifications without interrupting the flow of your content. They appear as numbered superscript links that jump to referenced text collected at the bottom of the page.

Define a footnote using square brackets with a caret and numbered reference:

```
Here is a footnote[^1] with some additional text after it.

[^1]: My reference.
```

Other common Markdown features

Starlight supports all other Markdown authoring syntax, such as lists and tables. See the [Markdown Cheat Sheet from The Markdown Guide](#) for a quick overview of all the Markdown syntax elements.

Advanced Markdown and MDX configuration

Starlight renders Markdown and MDX using Astro's configurable [markdown.processor](#). You can add support for custom syntax and behavior by passing plugins and options to your configured processor. See ["Markdown processor plugins"](#) in the Astro docs to learn more.

Markdoc

Starlight supports authoring content in Markdoc using the experimental [Astro Markdoc integration](#) and the Starlight Markdoc preset.

Create a new project with Markdoc

Start a new Starlight project with Markdoc pre-configured using `create astro`:

npm

pnpm

Yarn

```
npm create astro@latest -- --template starlight/markdoc
```

Add Markdoc to an existing project

If you already have a Starlight site and want to add Markdoc, follow these steps.

- 1 Add Astro's Markdoc integration:

npm

pnpm

Yarn

```
npx astro add markdoc
```

- 2 Install the Starlight Markdoc preset:

npm

pnpm

Yarn

```
npm install @astrojs/starlight-markdoc
```

3

Create a Markdoc configuration file at `markdoc.config.mjs` and use the Starlight Markdoc preset:

```
import { defineMarkdocConfig } from '@astrojs/markdoc/config';
import starlightMarkdoc from '@astrojs/starlight-markdoc';

export default defineMarkdocConfig({
  extends: [starlightMarkdoc()],
});
```

To learn more about the Markdoc syntax and features, see the [Markdoc documentation](#) or the [Astro Markdoc integration guide](#).

Configuring the Markdoc preset

The `starlightMarkdoc()` preset accepts the following configuration options:

headingLinks

type: boolean

default: true

Controls whether or not headings are rendered with a clickable anchor link. Equivalent to the [markdown.headingLinks](#) option, which applies to Markdown and MDX files.

```
export default defineMarkdocConfig({
  // Disable the default heading anchor link support
  extends: [starlightMarkdoc({ headingLinks: false })],
});
```

